Listing 4.10 contains a simple template for your view. Create RiffMates/templates/musicians.html, and add content similar to the listing's.

Listing 4.10 Template for the `musicians` view

```
<!-- RiffMates/templates/musicians.html -->
{% extends "base.html" %}

{% block title %}
  {{block.super}}: Musician Listing
{% endblock %}

{% block content %}
  <h1>Musicians</h1>

  <ul>
    {% for musician in musicians %}
      <li> <a href="{% url 'musician' musician.id %}">
        {{musician.last_name}}, {{musician.first_name}}
      </a> </li>
    {% empty %}
      <li> <i>No musicians in the database</i> </li>
    {% endfor %}
  </ul>

{% endblock content %}
```

The {% url %} tag points to the musician view and passes in the ID as an argument.

With the view written, the route registered, and a template defined, you're all set to visit your musician's listing page. Start your development server, and go to http://localhost:8000/bands/musicians/. The output should be similar to figure 4.8.

When you have more content in your database, the listing page will become cumbersome. There are a few ways of dealing with this; one is to limit the number of results in a page and support multiple pages.

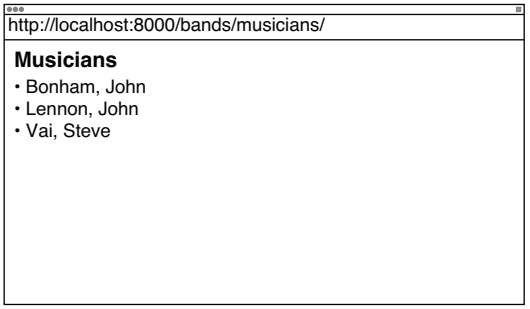


Figure 4.8 Musician listing page

4.6.3 Using query parameters for pagination

It is bad practice to support an unlimited number of objects in a query. Not only does this become information overload for your users, but it can be detrimental to your database. Django provides a utility to carve your queries up into pieces, allowing you to show a subset on the page. The `Paginator` class takes an iterator of objects and a limiter. The iterator can be a `QuerySet`, so you can pass the result of an object query directly to a `Paginator`. The limiter specifies the maximum number of objects per page.